



PariPari

Tracciare e dividere le spese tra amici, riducendo al minimo le transazioni. Anche offline.

Documentazione di progetto

Gruppo	Ctrl+C Ctrl+V
Componenti	Fabrizi Molodduwa Quiroz Andri 928604
	Gnoni Riccardo 907420
	Scotti Nicolò 916163
Repository	github.com/AndriMolo/PariPari
Piattaforma	Android · Java · Room · Firebase · ML Kit · Material 3

Indice

Introduzione	3
Strumenti di lavoro	3
1. Funzionalità	4
1.1 Account e profilo	4
1.2 Schede e inviti	5
1.3 Dettaglio scheda	5
1.4 Spese e quote	6
1.5 Saldi, rimborsi e pagamenti	6
1.6 Notifiche	7
1.7 Convertitore valuta	7
2. Architettura	8
2.1 Struttura dei package	9
2.2 Modello dei dati	10
2.3 Calcolo dei saldi	10
3. Design	11
3.1 Layouts	11
3.2 Palette e tipografia	15
Conclusioni	16
Punti di forza	16
Ostacoli incontrati	16
Sviluppi futuri	17

Introduzione

PariPari è un'app Android per tracciare e dividere le spese tra amici. Quando in un gruppo — coinquilini, compagni di viaggio, colleghi — persone diverse anticipano importi diversi, alla fine ognuno si ritrova a dover restituire frazioni di denaro a più persone. PariPari tiene il conto di chi ha pagato cosa e semplifica i conti riducendo al minimo le transazioni necessarie a chiudere i debiti.

L'app si fonda su quattro scelte di progetto:

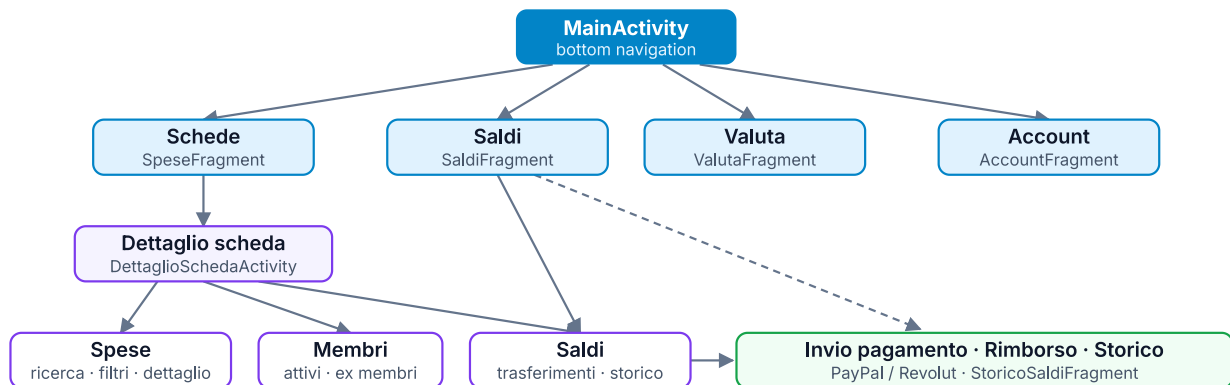
1. **Offline-first.** Ogni dato è scritto prima nel database locale Room; la sincronizzazione con Cloud Firestore avviene in background quando rete e account sono disponibili, e gli altri membri ricevono una notifica a ogni movimento.
2. **Pareggio dei saldi.** Le spese, anche in valute diverse, vengono ricondotte alla valuta del gruppo; un algoritmo greedy trasforma i bilanci netti nell'elenco dei rimborsi da fare: al più $N - 1$ per N membri.
3. **Inserimento senza attrito.** Scansione OCR dello scontrino, calcolatrice inline nell'importo, categoria suggerita dal titolo, divisione equa, per percentuali o per parti.
4. **Pagamenti in un tap.** I debiti si saldano con link PayPal.me e Revolut.me già compilati con importo e valuta; i rimborsi restano tracciati come transazioni a sé.

Strumenti di lavoro

- **Android Studio** — IDE ufficiale. Java (compatibilità 11), Gradle con Kotlin DSL e version catalog, ViewBinding, `minSdk 30 / targetSdk 37`.
- **GitHub** — repository `github.com/AndriMolo/PariPari` per versionamento e coordinamento del team, con commit incrementali per funzionalità sul branch `main`.
- **Room 2.6.1** — SQLite locale: 4 entità, DAO con query reattive LiveData, database alla versione 9 con cinque migrazioni esplicite, scritture su un pool di 4 thread.
- **Firestore (BoM 33.10.0)** — *Authentication* (email/password, Google Sign-In, ospite anonimo con account linking); *Cloud Firestore* (collezioni `users`, `groups`, `participants`, `expenses`, `shares`, regole in `firestore.rules`); *Storage* (avatar, icone dei gruppi, foto degli scontrini); *Cloud Messaging* (canale notifiche e token di dispositivo); *Hosting* (pagina web di invito con App Links).
- **Google ML Kit Text Recognition** — OCR degli scontrini eseguito sul dispositivo, senza inviare immagini a server esterni.
- **ZXing** — generazione dei QR code di invito, con il logo dell'app al centro.
- **Frankfurter API** — tassi di cambio della Banca Centrale Europea, pubblici e senza chiave.
- **Material Components 1.14 (Material 3)** — tema light/dark, BottomSheet, Chip, Card, Snackbar, MaterialDatePicker.
- **JUnit 4** — 11 classi di test unitari (49 test): codici invito, QR, conversione valuta, calcolatrice, categorie, ripartizione delle quote, ordinamento dei membri, anteprima del gruppo.

1. Funzionalità

L'app è organizzata in quattro schermate raggiunte dalla barra di navigazione inferiore di `MainActivity` — **Schede**, **Saldi**, **Valuta**, **Account** — più la schermata di dettaglio di una scheda (`DettaglioSchedaActivity`), a sua volta divisa in tre sezioni.



Mappa di navigazione delle schermate

1.1 Account e profilo AccountFragment · AccountGuestFragment · AccountProfileFragment ·

AccountViewModel

- **Modalità ospite.** Al primo avvio l'app crea un account anonimo Firebase: tutte le funzioni sono subito disponibili e il nome locale è modificabile.
- **Registrazione, accesso e Google.** Pulsanti separati "Crea account" e "Accedi" aprono il form (email validata, password di almeno 6 caratteri), affiancati da "Accedi con Google". Chi era ospite viene promosso ad account registrato con *account linking* (`linkWithCredential`): schede e spese create da ospite restano sue.
- **Verifica e recupero.** Email di verifica con polling ogni 2,5 secondi che porta il badge da "Email non verificata" a "Email verificata"; reinvio della verifica e reset della password via email.
- **Profilo.** Avatar scelto dalla galleria, ridimensionato e compresso in app e caricato su Firebase Storage (`avatars/{uid}.jpg`); nome propagato a tutte le schede (`aggiornaNomeUtenteInTuttiIGruppi`); cambio email con riautenticazione.
- **Pagamenti e preferenze.** Handle PayPal e Revolut propagati ai gruppi; valuta predefinita per le nuove schede, lingua (applicata a caldo) e tema chiaro, scuro o di sistema.

1.2 Schede e inviti SpeseFragment • SpeseViewModel • NuovaSchedaBottomSheet •

UniscitiSchedaBottomSheet

- **Elenco.** Le schede in ordine di ultimo aggiornamento, con icona del gruppo, titolo, data, valuta e numero di membri (conteggi da un'unica query aggregata); empty state quando non ce ne sono. Lo stato della schermata è un oggetto immutabile (`SpeseUiState`).
- **Nuova scheda.** BottomSheet con la valuta preimpostata sulla preferenza dell'utente; il creatore compare come primo membro "(io)", non modificabile in questa fase, e gli altri si aggiungono digitando il nome, come chip rimovibili.
- **Unisciti con codice.** Inserendo il codice a 6 caratteri (o aprendo link e QR code) l'app mostra l'anteprima dei membri della scheda: quelli già associati a un account sono in grigio, quelli inseriti "a mano" sono selezionabili per **subentrare** e diventarne il titolare; in coda c'è sempre l'opzione per aggiungersi come nuovo membro. La scheda viene poi importata in Room e messa in ascolto in tempo reale.
- **Importa da CSV.** Un file CSV esportato da PariPari ricrea scheda, membri, spese e quote sul dispositivo.
- **Gesti.** Pressione lunga e trascinamento per riordinare; swipe a sinistra per eliminare con Snackbar "Annulla", bloccato da `haSaldiInSospeso()` se l'utente ha ancora crediti o debiti aperti.

1.3 Dettaglio scheda DettaglioSchedaActivity • DettaglioSchedaFragment •

DettaglioSchedaViewModel

- **Toolbar.** Ricerca as-you-type; esportazione in **CSV** (UTF-8 con BOM) o **PDF** con riepilogo KPI, bilanci dei membri e tabella delle spese, condivisi via `FileProvider`; **codice gruppo** con QR code e link `paripari-app-2026.web.app/join?code=...`, che apre l'app direttamente o, dal browser, una pagina web di invito; rinomina; icona personalizzata del gruppo; **Lascia scheda**, che rende il membro un "ex membro" (riattivabile) e, se esce il creatore, passa la proprietà al successivo.
- **Sezione Spese.** Elenco raggruppato per data con chip per categoria; un tap apre il dettaglio (`DettaglioSpesaFragment`) con foto dello scontrino, modifica ed eliminazione.
- **Sezione Saldi.** Trasferimenti calcolati da `CalcolatoreSaldi`, bilancio netto di ogni membro e accesso allo storico.
- **Sezione Membri.** Attivi in cima con il capogruppo per primo, ex membri in una sezione a comparsa; modifica del nome inline (del proprio, o di tutti per il capogruppo); rimozione consentita al capogruppo o su di sé e bloccata se il membro ha spese o debiti (`haSpese0Debiti`).

1.4 Spese e quote BaseSpesaFragment · NuovaSpesaFragment · ModificaSpesaFragment ·

SpesaUiHelper

- **Campi.** Titolo; importo con calcolatrice inline (12,50+8 diventa 20,50 alla conferma); valuta della singola spesa, anche diversa da quella del gruppo; data da `MaterialDatePicker`; categoria; pagatore scelto tra i membri.
- **Categoria suggerita.** Predefinita "Altro"; mentre si digita il titolo, `CategoriaUtil` propone la categoria ("Esselunga" → Spesa, "Benzina" → Trasporti) e l'emoji corrispondente. Nove categorie: Cibo, Spesa, Trasporti, Alloggio, Svago, Shopping, Bar & Aperitivi, Salute, Altro.
- **Scontrino.** Foto o immagine dalla galleria: l'OCR di ML Kit, eseguito sul dispositivo, estrae totale, data ed esercente e precompila il form; l'immagine compressa viene caricata su Firebase Storage e resta allegata alla spesa.
- **Divisione.** Equa tra i partecipanti selezionati, per percentuali o per parti, con auto-bilanciamento delle quote residue e feedback aptico sulle azioni.
- **Consistenza storica.** Se il pagatore o uno dei partecipanti ha lasciato il gruppo, la modifica è bloccata con un avviso: i saldi passati restano corretti.

1.5 Saldi, rimborsi e pagamenti SaldiFragment · InvioPagamentoBottomSheet ·

StoricoSaldiFragment · DettaglioRimborsoFragment

- **Riepilogo.** Due card, "Ti devono" (verde) e "Devi tu" (rosso), con i totali su tutte le schede e un tap per filtrare; sotto, i bilanci netti. Le spese in valuta diversa sono convertite nella valuta del gruppo con i tassi BCE.
- **Pagamento.** Toccando una voce si apre il foglio di pagamento, che riconosce se l'utente è debitore o creditore e, se il creditore ha un handle, genera il link PayPal.me o Revolut.me con importo e valuta: apribile, copiabile o condivisibile.
- **Rimborsi.** "Segna come saldato", o un rimborso inserito a mano, registra una transazione di tipo *Rimborso* — mittente, destinatario, importo, valuta, data, nota — con schermate di dettaglio e modifica; è esclusa dal totale speso e riduce il debito in locale e su Firestore.
- **Storico.** Schermata dedicata con filtro per scheda; un pareggio si può annullare, salvo quando coinvolge un membro uscito.

1.6 Notifiche FirestoreSyncManager • PariPariMessagingService

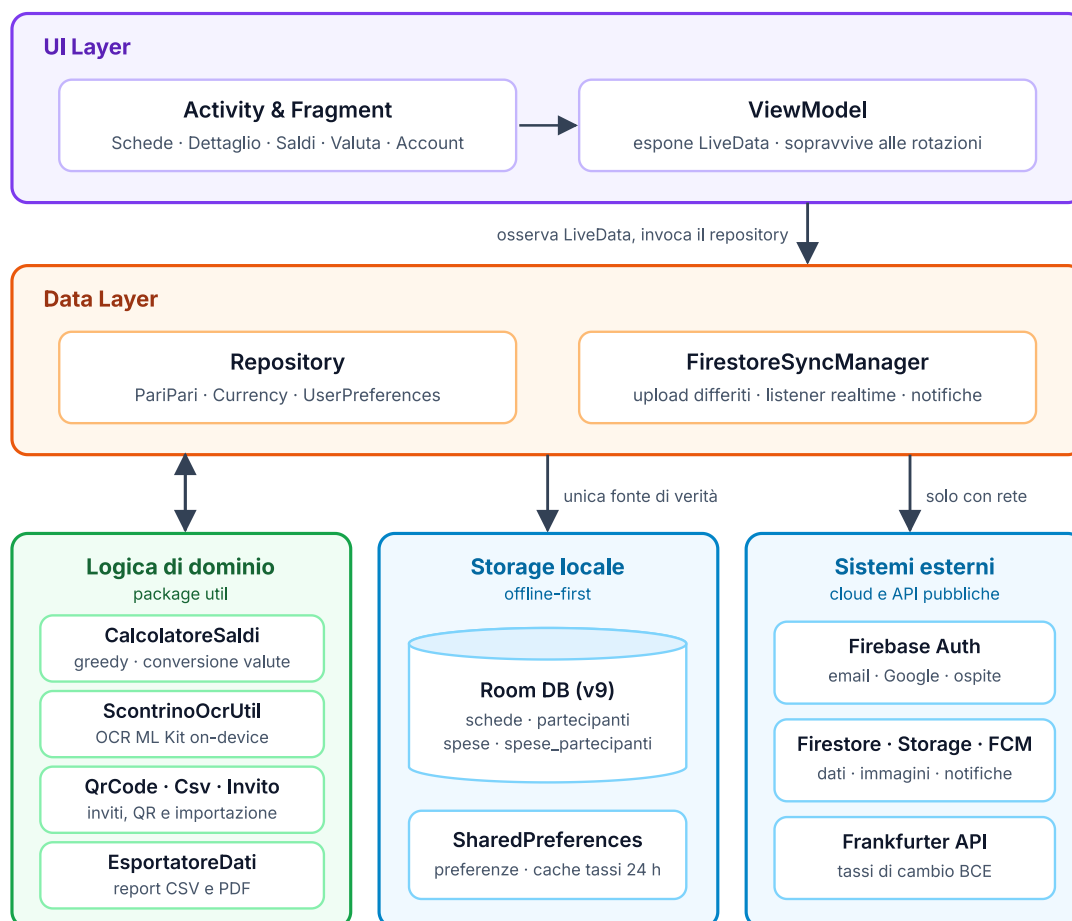
- **Movimenti del gruppo.** Quando un altro membro aggiunge o modifica una spesa, o segna un rimborso, il listener Firestore che riceve la modifica genera una notifica nativa ("Nuova spesa in ...", con importo e propria quota); il tap apre la scheda interessata.
- **Permessi e canale.** All'avvio l'app chiede il permesso `POST_NOTIFICATIONS` (Android 13+) e registra il canale di notifica.
- **Firestore Cloud Messaging.** Il servizio riceve eventuali messaggi push e registra il token del dispositivo in `users/{uid}/fcm_tokens`: la base per notifiche inviate dal server anche ad app chiusa.

1.7 Convertitore valuta ValutaFragment • ValutaViewModel • CurrencyRepository

- **Tassi BCE.** `CurrencyRepository` interroga `api.frankfurter.dev` al massimo una volta ogni 24 ore e salva i tassi in `SharedPreferences`; senza rete usa la cache o i tassi di fallback integrati. 30 valute supportate, con bandiera e nome. Gli stessi tassi servono a convertire le spese multivaluta nei saldi.
- **Conversione as-you-type.** Risultato e tasso applicato si aggiornano mentre si digita; il pulsante "Inverti" scambia le due valute.
- **Selettore.** `BottomSheet` con le valute frequenti in testa e ricerca testuale sul catalogo completo; lo stesso selettore è usato per la valuta di una scheda o di una spesa.

2. Architettura

PariPari segue il pattern **MVVM** raccomandato dalle linee guida Android, con un **Repository** che incapsula le sorgenti dati. Le classi vivono nel namespace `com.example.pari pari app`, diviso in `data` (persistenza locale, sincronizzazione remota, repository), `ui` (View e ViewModel), `util` (algoritmi e classi di supporto) e `service` (notifiche push).



Architettura a layer: MVVM + Repository

Offline-first e sincronizzazione

Room è l'unica fonte di verità della UI: i Fragment osservano i `LiveData` prodotti dai DAO e non leggono mai direttamente da Firestore. Ogni riga porta un `sync_status` (`SYNCED`, `PENDING_INSERT`, `PENDING_UPDATE`, `PENDING_DELETE`). `FirestoreSyncManager` osserva la rete con `NetworkConnectivityMonitor` e lo stato di autenticazione: quando entrambi sono disponibili invia le righe pendenti, aggancia listener snapshot su `groups` e sulle sue sotto-collezioni e scrive in Room le modifiche degli altri membri — che diventano anche notifiche. Quando l'app va in background la sincronizzazione si sospende e riparte al ritorno in primo piano (`ActivityLifecycleCallbacks` in `PariPariApplication`), per risparmiare batteria e rete. Tutte le scritture locali passano dal pool `databaseWriteExecutor`, tenendo libero il main thread.

2.1 Struttura dei package

2.1.1 data.local — database Room e DAO

`AppDatabase` definisce il database (versione 9, migrazioni da 4→5 a 8→9, pool di 4 thread) ed espone `SchedaDao`, `PartecipanteDao` e `SpesaDao`: query reattive, conteggi aggregati, spese con dettagli via LEFT JOIN, transazioni spesa+quote, storico dei saldi.

2.1.2 data.model — entità e DTO

Le entità Room `Scheda`, `Partecipante` (con proprietà, ordinamento e riconoscimento dell'utente corrente), `Spesa` e `SpesaPartecipante`; i DTO per liste e saldi (`SpesaConDettagli`, `TrasferimentoSaldo`, `RisultatoSaldi`, ...) e `MembroGruppoPreview` per l'anteprima all'ingresso; le costanti `SyncStatus`.

2.1.3 data.remote — Firestore

`FirestoreSyncManager` concentra il lato cloud: upload differiti di schede, membri, spese e quote, cancellazioni, listener in tempo reale, risoluzione dei codici invito, notifiche locali dai cambiamenti ricevuti.

2.1.4 data.repository — Repository

`PariPariRepository` è la facciata unica verso i ViewModel: transazioni Room, delega al sync manager, ingresso con codice e subentro, importazione CSV, upload di avatar e icone. `CurrencyRepository` scarica e mette in cache i tassi. `UserPreferencesRepository` gestisce valuta, lingua, tema, handle di pagamento e l'associazione utente↔partecipante per scheda.

2.1.5 service — PariPariMessagingService

Estende `FirebaseMessagingService`: registra il token in Firestore, crea il canale di notifica e mostra le notifiche native con il deep link alla scheda.

2.1.6 ui.viewmodel — ViewModel

`SpeseViewModel` (schede, ingresso e subentro, rimborsi, import CSV, stato `SpeseUiState`), `DettaglioSchedaViewModel` (spese filtrate, saldi, membri), `AccountViewModel` (email/Google, account linking, verifica, reset password), `ValutaViewModel`. Estendono `AndroidViewModel` ed espongono solo `LiveData`.

2.1.7 ui.view — Activity, Fragment, BottomSheet e Adapter

`MainActivity` (splash, permesso notifiche, deep link) e `DettaglioSchedaActivity`; i Fragment delle quattro sezioni (`Account` con i figli `Guest` e `Profile`), `DettaglioScheda`, `DettaglioSpesa`, `DettaglioRimborso`, `StoricoSaldi` e `BaseSpesa`, da cui derivano `NuovaSpesa`, `ModificaSpesa` e `ModificaRimborso`; sei `BottomSheet` (nuova scheda, unisciti, pagamento, valuta, avatar, icona gruppo); nove `Adapter`; `SpesaUiHelper` per validazione e bilanciamento delle quote.

2.1.8 util — algoritmi e supporto

Algoritmi: `CalcolatoreSaldi`, `CategoriaUtil`, `CodiceInvitoUtil`, `CalcolatriceEspressioniUtil`. Integrazioni: `ScontrinoOcrUtil` (ML Kit e upload), `QrCodeUtil` (ZXing), `ImportatoreCsvUtil`, `EsportatoreDati` (CSV e PdfDocument), `ImageLoaderUtil` (LruCache). Interfaccia: `AppSnackbar`, `HapticUtil`, `KeyboardUtil`, `ImportoUtil`, `SpesaUtil`, `DecimalDigitsInputFilter`, `NetworkConnectivityMonitor`.

2.2 Modello dei dati

Quattro tabelle Room, con ID `UUID` generati dal client così da poter creare dati offline senza conflitti. Ogni riga porta il proprio `sync_status`.

schede	partecipanti
PK id titolo · descrizione · icona_url (v9) valuta_predefinita · creatore_id data_creazione · data_aggiornamento codice_invito (v6) · sync_status	PK id FK scheda_id → schede nome · email · photo_url (v9) user_id · previous_user_id · stato (v8) paypal_handle · revolut_handle (v7) sync_status
spese	spese_partecipanti
PK id FK scheda_id → schede titolo · importo · valuta · data_spesa categoria · pagato_da_id → partecipanti scontrino_url · sync_status	PK,FK spesa_id → spese PK,FK partecipante_id → partecipanti quota · quota_pagata (v5) sync_status

Una scheda ha N partecipanti e N spese; ogni spesa è pagata da un partecipante e ripartita tramite la tabella ponte, che conserva quota dovuta e quota già saldata. Un partecipante può essere legato a un account (`user_id`) o esistere solo come nome inserito da altri: chi entra con il codice può subentrargli (`previous_user_id`) e lo `stato` distingue attivi, usciti, ospiti e archiviati. Su Firestore le stesse strutture sono `groups/{id}` con le sotto-collezioni `participants`, `expenses` ed `expenses/{id}/shares`; `users/{uid}` tiene profilo e token FCM; le immagini stanno su Storage.

Migrazioni. 4→5 `quota_pagata`; 5→6 `codice_invito`; 6→7 handle PayPal e Revolut; 7→8 `user_id`, `previous_user_id`, `stato`; 8→9 `icona_url` e `photo_url`. Ogni evoluzione preserva i dati già sul dispositivo.

2.3 Calcolo dei saldi

`CalcolatoreSaldi` lavora in tre passi, per ogni scheda, dopo aver convertito nella valuta del gruppo le spese registrate in altre valute:

1. **Bilancio netto** di ogni membro: somma degli importi che ha anticipato, meno le quote residue che deve (`quota - quota_pagata`); i rimborsi già registrati riducono sia il debito sia il credito.
2. **Separazione** in debitori (bilancio negativo) e creditori (bilancio positivo), con arrotondamento al centesimo.
3. **Accoppiamento greedy**: il primo debitore paga al primo creditore il minimo tra debito e credito; chi si azzerava avanza. Ogni trasferimento chiude almeno una posizione, quindi i rimborsi sono al più $N - 1$ per N membri.

Esempio: Anna ha anticipato 90 €, Bruno 30 € e Carla 0 € su spese divise equamente (40 € a testa). Bilanci: Anna +50, Bruno -10, Carla -40. Invece di tre rimborsi incrociati bastano due trasferimenti: Bruno → Anna 10 € e Carla → Anna 40 €.

3. Design

L'interfaccia segue **Material 3** — Card, Chip, BottomSheet, BottomNavigation, Snackbar — con una palette dedicata per tema chiaro e scuro. Layout XML con ViewBinding, edge-to-edge, empty state per ogni lista vuota e feedback aptico sulle azioni.

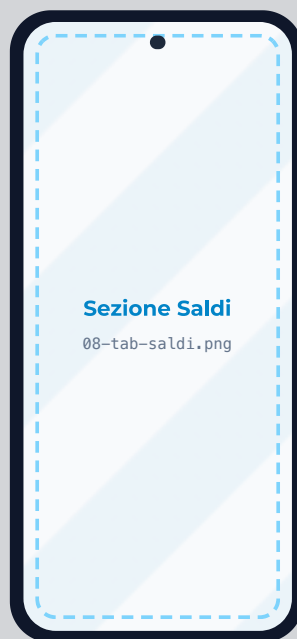
3.1 Layouts



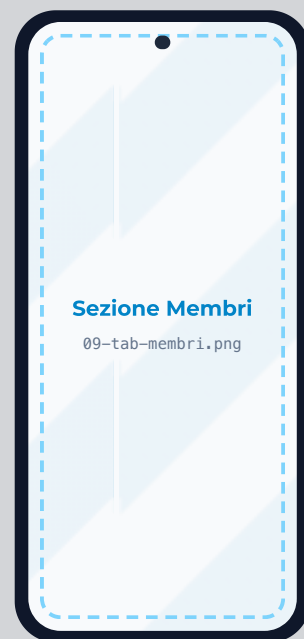
Dettaglio scheda



Spese, ricerca e filtri

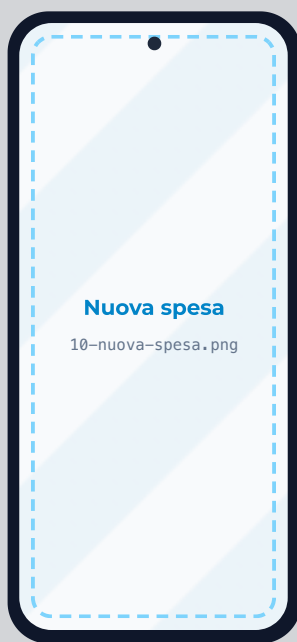


Saldi e bilanci



Membri attivi ed ex membri

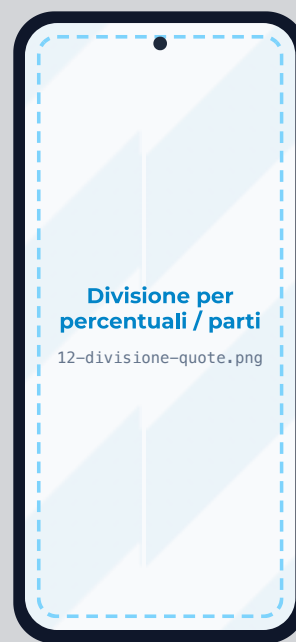
Inserimento spese



Nuova spesa



Scansione dello scontrino



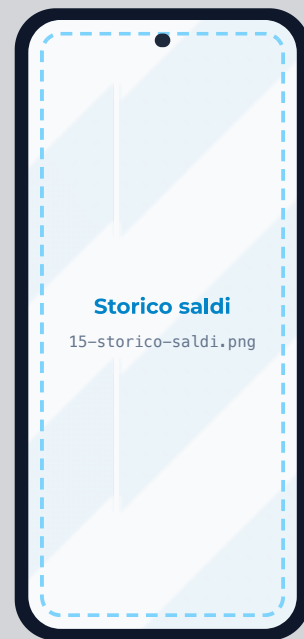
Divisione per percentuali o parti

Saldi e pagamenti

I miei saldi



Invio pagamento



Storico dei saldi

Convertitore valuta

Conversione as-you-type

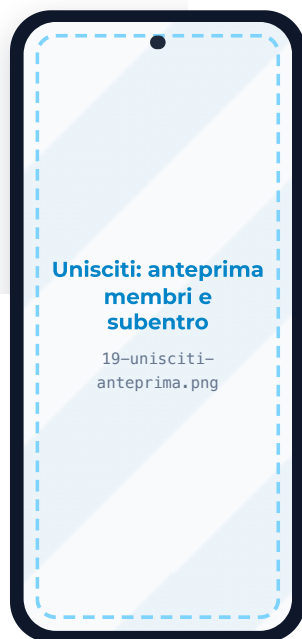


Risultato e tasso BCE



Selettore valuta

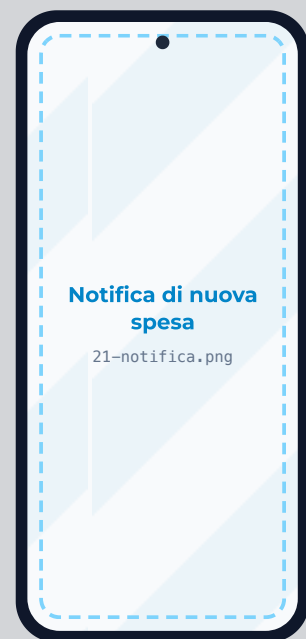
Ingresso nel gruppo, rimborsi e notifiche



Unisciti con anteprima dei membri



Rimborso



Notifica di nuova spesa

3.2 Palette e tipografia

La palette è costruita sui ruoli semantici di Material 3. Il tema chiaro usa un ciano profondo su fondi neutri freddi; quello scuro passa a toni neon su sfondo quasi nero. Verde, rosso e arancione sono riservati agli stati contabili: credito, debito, avviso.

#0284C7 #7C3AED #F8FAFC #0F172A #15803D #B91C1C					
Ruolo	Risorsa	Chiaro	Scuro	Uso	
Primario	<code>md_theme_primary</code>	 #0284C7	 #00E5FF	Toolbar, pulsanti, accenti attivi	
Contenitore primario	<code>md_theme_primaryContainer</code>	 #E0F2FE	 #003846	Badge circolari, pillole valuta	
Secondario	<code>md_theme_secondary</code>	 #7C3AED	 #A855F7	Accenti secondari	
Sfondo	<code>md_theme_background</code>	 #F8FAFC	 #0B0E17	Sfondo delle schermate	
Superficie	<code>md_theme_surface</code>	 #FFFFFF	 #101422	Card, BottomSheet, dialog	
Credito	<code>credit_green</code>	 #15803D	 #00FF87	"Ti devono", saldo chiuso	
Debito	<code>debt_red</code>	 #B91C1C	 #FF2A6D	"Devi tu", eliminazione	
Avviso	<code>warning_orange</code>	 #D97706	 #FFB800	Email non verificata	

Tipografia e stili

- **Font.** Nessun font incorporato: l'app usa la scala tipografica di Material 3 con il carattere di sistema (Roboto), per coerenza con la piattaforma e massima leggibilità delle cifre.
- **Card** (`Widget_PariPari_CardItem`): angoli a 16dp, elevazione 1dp, bordo di 1dp sul colore outline, sfondo `surfaceContainerLow`.
- **Badge circolari** (`Widget_PariPari_CircleIconBadge`): raggio 24dp su `primaryContainer` per avatar, stati e valute.
- **BottomSheet e Snackbar.** Fogli con angoli superiori a 28dp e maniglia 32×4dp; Snackbar a pillola (20dp) su `surfaceContainerHighest`, con animazione a scorrimento, al posto dei toast.
- **Icona e splash.** Icona adattiva con il glifo "PP" — un più e un meno, le due facce di ogni spesa condivisa — su sfondo bianco, con variante monocromatica per Android 13+; lo stesso glifo anima lo splash screen di sistema.

Conclusioni

PariPari copre l'intero ciclo di una spesa condivisa: fotografarla o registrarla, ripartirla, calcolare chi deve cosa a chi, saldare e avvisare gli altri. L'architettura MVVM con Repository e l'approccio offline-first hanno retto bene alla crescita del progetto: le funzionalità aggiunte nella seconda fase — OCR, inviti web e QR, rimborsi, notifiche, immagini — si sono innestate senza cambiare l'impianto.

Punti di forza

1. **Funziona sempre.** Nessuna operazione dipende dalla rete: Room risponde subito e la sincronizzazione recupera il pregresso quando torna la connessione, in modo trasparente per l'utente e con la pausa in background a tutela della batteria.
2. **Conti chiari.** Bilanci netti in un'unica valuta anche con spese miste, trasferimenti minimi, rimborsi tracciati come transazioni con uno storico annullabile e blocchi che impediscono di lasciare debiti orfani.
3. **Inserimento rapido.** Scontrino fotografato e letto sul dispositivo, calcolatrice nell'importo, categoria suggerita dal titolo, divisione per percentuali o parti che si bilancia da sola.
4. **Dal saldo al pagamento in un gesto.** Il link PayPal.me o Revolut.me precompilato chiude il cerchio, e la notifica avvisa il gruppo senza scambiarsi messaggi.

Ostacoli incontrati

1. **Sincronizzazione bidirezionale.** Tenere allineati Room e Firestore senza duplicare scritture ha richiesto uno stato per riga e una gestione attenta del ciclo di vita dei listener, oggi isolata in `FirestoreSyncManager` e sospesa quando l'app non è in primo piano.
2. **Identità dei membri.** Un partecipante può nascere come semplice nome digitato da un altro utente e solo dopo corrispondere a un account: il modello con `user_id`, `previous_user_id` e `stato`, il subentro all'ingresso e l'account linking per gli ospiti hanno risolto i casi di "chi sono io" in ogni scheda.
3. **Integrità dei saldi.** Eliminare una scheda o un membro con pendenze aperte corrompeva i conti: i controlli `hasaldiInSospeso()` e `haspese0Debiti()` bloccano le operazioni pericolose, la modifica di una spesa è inibita se un partecipante coinvolto è uscito e i membri usciti restano come "ex" invece di sparire.
4. **Evoluzione dello schema.** Pareggi parziali, codici invito, handle di pagamento, subentro e immagini sono arrivati in fasi successive: cinque migrazioni Room esplicite hanno permesso di aggiungerli senza perdere i dati sul dispositivo.